

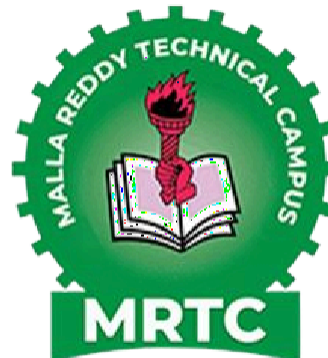


MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



DESIGN AND ANALYSIS OF ALGORITHMS

QUESTION BANK





MODULE – I

S.No	Short Answer Questions	Page no
1	Define an Algorithm.	4
2	Explain the characteristics of an Algorithm.	4
3	What is the difference between Time Complexity and Space Complexity?	4
4	What is the Divide and Conquer Technique?	4
5	How many multiplications are required in Strassen's algorithm?	4
6	What is a pivot element in Quick Sort?	5

S.No	Long Answer Questions	Page no
1	Explain the Tower of Hanoi problem in detail. Write the recursive Algorithm for solving it and describe the steps involved in moving the disks from source to destination.	6
2	Explain performance analysis of algorithms. Discuss time complexity and space complexity with suitable algorithm examples.	9
3	What is Binary Search? Explain the algorithm with a suitable example and analyze its time complexity.	11
4	Explain Strassen's Matrix Multiplication algorithm with an Example.	12
5	Explain the Divide and Conquer strategy with an Algorithm and discuss its applications.	16
6	Explain Merge sort Algorithm with an example and analyze its time complexity.	17



MODULE – II

S.No	Short Answer Questions	Page no
1	What is Hamiltonian cycle?	21
2	Write an Algorithm for Simple Union Operation?	21
3	What is the state-space tree in backtracking?	21
4	Define the Find operation in a disjoint-set data structure.	21
5	How is a Heap Implemented?	21
6	What is the Sum of Subsets Problem?	22

S.No	Long Answer Questions	Page no
1	Explain Disjoint Set Operations. Describe Union and Find algorithms with examples.	23
2	Explain the N-Queens Problem using Backtracking. Illustrate the solution for a 4-Queens problem.	27
3	Describe the general method of Backtracking with an algorithm and write its applications.	31
4	Generate the state space tree and find the solutions for the subset sum for $N=4$, $M=50$, $W=\{10,20,30,40\}$ using backtracking approach.	32
5	Explain the Graph Coloring problem and solve it using the Backtracking technique.	35

6	What is a Hamiltonian Cycle? Explain the Backtracking solution for finding Hamiltonian Cycles in a graph.	39
---	---	----



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



MODULE – I

Short Answer Questions

1. Define an Algorithm.

An Algorithm is a finite sequence of instructions, each of which has a clear meaning and can be performed with a finite amount of effort in a finite length of time.

2. Characteristics of an Algorithm.

Remember it like: "I DEFE" — Input, Definiteness, Effectiveness, Finiteness, Extra(Output)

Answer: (5 points)

Input – takes 0 or more values from outside -

Output – gives at least 1 result

Definiteness – each step is clear, no confusion

Finiteness – it must stop after some steps (not run forever)

Effectiveness – steps are simple enough to do by hand

3. What is the difference between Time Complexity and Space Complexity?

Time complexity measures how much time an algorithm takes based on input size. Space complexity measures how much memory an algorithm uses. Both help evaluate and compare algorithm efficiency.

4. What is the Divide and Conquer Technique?

Divide and Conquer is an algorithm design technique in which a problem is repeatedly divided into smaller subproblems of the same type, each subproblem is solved independently, and their solutions are combined to obtain the final solution.

5. How many multiplications are required in Strassen's algorithm?

In conventional matrix multiplication of two 2×2 matrices, 8 multiplications are required.

In Strassen's Matrix Multiplication algorithm, the number of multiplications are reduced to 7 multiplications and 18 additions/subtractions.



6. What is a pivot element in Quick Sort?

A pivot element is the element selected from the array during the Quick Sort process to partition the array into two parts.

1. Elements smaller than the pivot are placed on its left side.
2. Elements greater than the pivot are placed on its right side.
3. After partitioning, the pivot is placed in its correct sorted position.



LONG ANSWERS

1. Explain the Tower of Hanoi problem in detail. Write the recursive Algorithm for solving it and describe the steps involved in moving the disks from source to destination.

The Towers of Hanoi is a mathematical puzzle used to explain the concept of recursion.

It consists of three pillars (rods):

- Source
- Auxiliary
- Destination

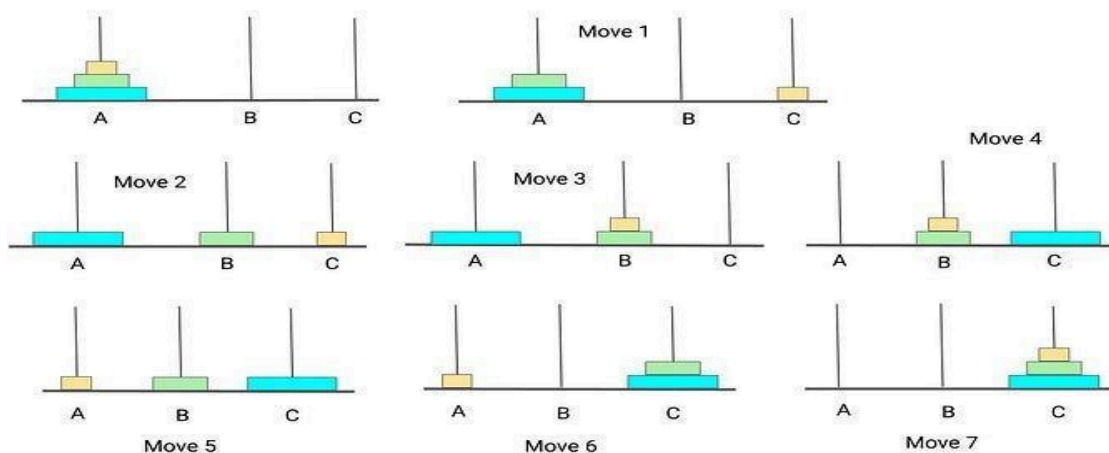
and three disks of different sizes placed on the Source rod in ascending order of size (smallest at the top, largest at the bottom).

Rules of the Problem

1. Only one disk can be moved at a time.
2. Only the top disk of any rod can be moved.
3. A larger disk cannot be placed on a smaller disk.

Objective

Move all three disks from the Source rod to the Destination rod using the Auxiliary rod, without changing the order of disks.





Steps for 3 Disks

1. Move disk 1 from Source → Destination
2. Move disk 2 from Source → Auxiliary
3. Move disk 1 from Destination → Auxiliary
4. Move disk 3 from Source → Destination
5. Move disk 1 from Auxiliary → Source
6. Move disk 2 from Auxiliary → Destination
7. Move disk 1 from Source → Destination

Total moves required = $2^3 - 1 = 7$ moves

Pseudocode for Tower of Hanoi:

Algorithm TOH(*n*, Source, Auxiliary, Destination)

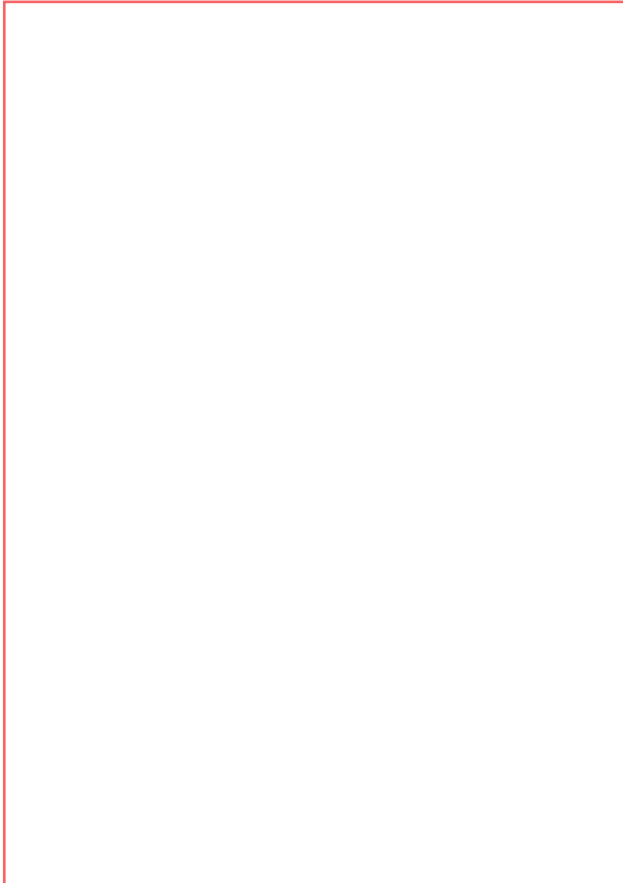
```
{
if (n > 1 ) then
{
TOH(n - 1, Source, Destination, Auxiliary)

write("Move disk ", n, " from ", Source, " to ", Destination)

TOH(n - 1, Auxiliary, Source, Destination)
}
}
```

Parameters

- **n** → Number of disks
- **A** → Source tower
- **B** → Destination tower
- **C** → Auxiliary (temporary) tower



Solution:

$$T(n) = 2^n - 1$$

For $n = 3$:

$$T(3) = 2^3 - 1$$

$$= 8 - 1$$

$$= 7 \text{ moves}$$

Recurrence Relation :

$$T(n) = 2T(n - 1) + 1$$

Complexity

Time Complexity: $O(2^n)$

Space Complexity: $O(n)$ (due to recursion stack)



2.Explain performance analysis of an algorithm.Discuss time complexity and space complexity with suitable algorithm example.

The performance of a program is the amount of computer memory and time needed to run a program. We use two approaches to determine the performance of a program. One is analytical, and the other experimental.

In performance analysis, we use analytical methods, while in performance measurement, we conduct experiments.

Performance analysis of an algorithm can be measured in two ways

- i)By calculating its time complexity
- ii)By calculating its space complexity

Time Complexity: The time needed by an algorithm expressed as a function of the size of a problem is called the time complexity of the algorithm. The time complexity of a program is the amount of computer time it needs to run to completion.

Space Complexity:

Space complexity is the total amount of memory required by an algorithm during its execution. It represents how much storage space an algorithm needs to solve a problem.

The memory may be required for:

- Variables
- Constants
- Arrays
- Recursion
- Dynamic memory allocation

Space Requirement Formula: $S(P) = C + S_p$

where

Fixed Part (C)

The fixed part is independent of input size.

It includes:

- Program instructions
- Constants
- Simple variables
- Fixed-size data structures

This memory remains unchanged regardless of the input.



Variable Part (SP)

The variable part depends on the size of the input.

It includes:

- Arrays
- Dynamic memory
- Recursion stack
- Variable-size data structures

As the input grows, this memory also grows.

Pseudocode

Algorithm SUM(A,n)

```
{  
sum ← 0  
  
for i ← 1 to n do  
    sum ← sum + A[i]  
  
return sum  
}
```

Time Complexity Analysis

Statement	Cost	Repetition	Total
sum ← 0	1	1	1
for loop condition	1	n+1	n+1
sum ← sum + A[i]	2	n	2n
return sum	1	1	1

$$T(n) = 1 + (n + 1) + 2n + 1T(n) = 3n + 3$$

Ignoring constants,

$$T(n) = O(n)$$



Space Complexity Analysis

Variables:

- Array A $\rightarrow$ n words
- n $\rightarrow$ 1 word
- i $\rightarrow$ 1 word
- sum $\rightarrow$ 1 word

Total space:

$$S(P) = n + 3$$

Hence,

$$S(P) = O(n)$$

3. What is Binary Search? Explain the algorithm with a suitable example and analyze its time complexity.

Binary Search : Binary Search is a searching technique used to find an element in a sorted array. It works by repeatedly dividing the search space into half.

Pseudocode for Binary Search:

```
int BinarySearch(int A[], int n, int key)
{
    int low=0, high=n-1;
    while(low<=high)
    {
        int mid=(low+high)/2;
        if(A[mid]==key) return mid;
        if(key<A[mid]) high=mid-1;
        else low=mid+1;
    }
    return -1;
}
```



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



Tracing of Binary Search

Given Array

$A = \{10, 20, 30, 40, 50, 60, 70\}$

Key = 50

Step 1:

Initialize:

low = 0

high = 6

Calculate middle element:

$mid = (low + high)/2$

$= (0 + 6)/2$

$= 3$

Therefore,

$A[3] = 40$

Compare:

$50 > 40$

Since the key is greater than the middle element, search the **right half**.

Update:

low = 4

high = 6

Step 2:

Calculate middle element:

$mid = (4 + 6)/2$

$= 5$

Therefore,

$A[5] = 60$

Compare:

$50 < 60$

Since the key is smaller than the middle element, search the **left half**.

Update:

low = 4

high = 4

Step 3:

Calculate middle element:

$mid = (4 + 4)/2$

$= 4$

Therefore,

$A[4] = 50$

Compare: $50 = 50$



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



The key element is found.

Return 4

Conclusion

The element 50 is found at index 4 after 3 iterations.

Time Complexity Analysis of Binary Search

Best Case

The best case occurs when the key element is found at the middle position in the very first comparison.

Example

$A = \{10, 20, 30, 40, 50, 60, 70\}$

Key = 40

Since the middle element itself is 40, only one comparison is required.

Time complexity:

Best case time complexity: $O(1)$

Average case time complexity: $O(\log n)$

Worst case time complexity: $O(\log n)$



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



4. Explain Strassen's Matrix Multiplication algorithm with an Example.

Strassen's Matrix Multiplication is a Divide and Conquer algorithm proposed by Volker Strassen for multiplying two matrices faster than the conventional method.

The ordinary matrix multiplication of two $n \times n$ matrices require:

$$O(n^3)$$

time complexity.

Strassen's algorithm reduces this to:

$$O(n^{\log_2 7}) O(n^{2.807})$$

which is faster for large matrices.

Divide and Conquer Strategy

Given two matrices A and B of order $n \times n$, divide each matrix into four submatrices of size $\frac{n}{2} \times \frac{n}{2}$.

$$A = \begin{bmatrix} A_{11} & A_{12} & A_{21} & A_{22} \end{bmatrix} B = \begin{bmatrix} B_{11} & B_{12} & B_{21} & B_{22} \end{bmatrix}$$

The resultant matrix is

$$C = \begin{bmatrix} C_{11} & C_{12} & C_{21} & C_{22} \end{bmatrix}$$

Conventional Method

Normally,

$$C_{11} = A_{11}B_{11} + A_{12}B_{21} \quad C_{12} = A_{11}B_{12} + A_{12}B_{22} \quad C_{21} = A_{21}B_{11} + A_{22}B_{21} \quad C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

This requires 8 multiplications.



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



Strassen's Improvement

Instead of 8 multiplications, Strassen computes only 7 multiplications.

Step 1: Compute Seven Products

$$\begin{aligned} P &= (A_{11} + A_{22})(B_{11} + B_{22}) \\ Q &= (A_{21} + A_{22})B_{11} \quad R = A_{11}(B_{12} - B_{22}) \quad S = A_{22}(B_{21} - B_{11}) \quad T = (A_{11} + A_{12})B_{22} \\ U &= (A_{21} - A_{11})(B_{11} + B_{12}) \\ V &= (A_{12} - A_{22})(B_{21} + B_{22}) \end{aligned}$$

Step 2: Compute Result Matrix

$$C_{11} = P + S - T + V \quad C_{12} = R + T \quad C_{21} = Q + S$$

$$C_{22} = P + R - Q + U$$

Example
Multiply

and

$$a = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$b = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Step 1: Identify Elements

$$A_{11} = 1, A_{12} = 2, A_{21} = 3, A_{22} = 4, B_{11} = 5, B_{12} = 6, B_{21} = 7, B_{22} = 8$$

Step 2: Calculate the Seven Products

$$P = (1 + 4)(5 + 8) = 5 \times 13 = 65$$

$$Q = (3 + 4)(5) = 7 \times 5 = 35$$

$$R = (1)(6 - 8) = 1 \times (-2) = -2, S = (4)(7 - 5) = 4 \times 2 = 8, T = (1 + 2)(8) = 3 \times 8 = 24$$

$$U = (3 - 1)(5 + 6) = 2 \times 11 = 22$$

$$V = (2 - 4)(7 + 8) = (-2) \times 15 = -30$$



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



Step 3: Compute Result Matrix

$$C_{11} = 65 + 8 - 24 - 30 = 19$$

$$C_{12} = -2 + 24 = 22, C_{21} = 35 + 8 = 43$$

$$C_{22} = 65 - 35 - 2 + 22 = 50$$

Final Result

$$C = [19 \ 22 \ 43 \ 50]$$

Recurrence Relation

$$T(n) = 7T\left(\frac{n}{2}\right) + O(n^2)$$

5. Explain the Divide and Conquer strategy with an algorithm and discuss its applications.

- Divide and Conquer is an algorithm design technique in which a problem is repeatedly divided into smaller subproblems of the same type, each subproblem is solved independently, and their solutions are combined to obtain the final solution.

- This technique is widely used to design efficient algorithms for sorting, searching, matrix multiplication, and many other problems.

Basic Steps

1. Divide

Break the given problem into smaller subproblems.

2. Conquer

Solve each subproblem recursively. If the subproblem is small enough, solve it directly.

3. Combine

Merge the solutions of the subproblems to obtain the solution to the original problem.

Control Abstraction of Divide and Conquer

The control abstraction for divide and conquer technique is DANDC(P), where P is the problem to be solved.

```
Algorithm MergeSort(A, low, high)
{
  if (low < high) then {
    mid := (low + high)/2;
    MergeSort(A, low, mid);
    MergeSort(A, mid+1, high);
    Merge(A, low, mid, high);
  }
}
```

SMALL (P) is a Boolean-valued function which determines whether the input size is small enough so that the answer can be computed without splitting. If this is so, function 'S' is invoked; otherwise, the problem 'p' is split into smaller subproblems. These subproblems p1, p2, . . . , pk are solved by recursive application of DANDC.

Recurrence Relation

The running time of Divide and Conquer algorithms is generally expressed as:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Where:

- **a** = Number of subproblems
- **n/b** = Size of each subproblem
- **f(n)** = Time required to divide and combine

Applications of Divide and Conquer:

1. Binary Search
2. Quick sort

- 3. Merge sort
- 4. Strassen's Matrix Multiplication

6. Explain Merge sort Algorithm with an example and analyze its time complexity.

Merge Sort :

Merge Sort is a divide-and-conquer sorting algorithm that divides the array into smaller parts, sorts them, and then merges them back together in sorted order.

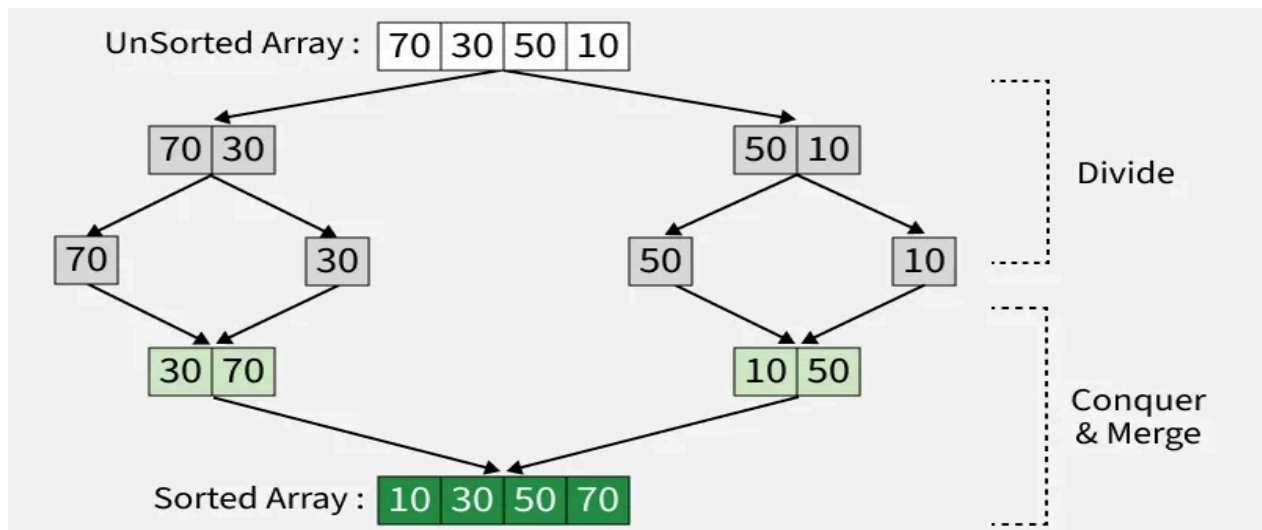
The array is split into smaller and smaller pieces until it is merged back together. And as the merging happens, values from each sub-array are compared so that the lowest value comes first.

Here's a step-by-step explanation of how merge sort works:

1. **Divide:** Divide the list or array recursively into two halves until it can no more be divided.
2. **Conquer:** Each subarray is sorted individually using the merge sort algorithm.
3. **Merge:** The sorted subarrays are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



Here, a problem is divided into multiple sub-problems. Each sub-problem is solved individually. Finally, sub-problems are combined to form the final solution.

Merge Sort Algorithm

```

{
  if (low < high) then
  {
    mid := (low + high)/2;

    MergeSort(A, low, mid);

    MergeSort(A, mid+1, high);

    Merge(A, low, mid, high);
  }
}

```



Merge [30 70] and [10 50]

Compare 30 and 10 → 10

Compare 30 and 50 → 30

Compare 70 and 50 → 50

Remaining → 70

Final Sorted Array:

[10 30 50 70]



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



Time Complexity Analysis of Merge Sort:

Recurrence Relation

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Case	Time Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



MODULE – II

Short Answer Questions

1. What is Hamiltonian cycle?

A Hamiltonian cycle is a cycle in a graph that visits every vertex exactly once and returns to the starting vertex.

Example:

In a graph with vertices A, B, C, and D, the path $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$ is a Hamiltonian cycle if each vertex is visited exactly once before returning to A. Not every graph contains a Hamiltonian cycle.

Q2. Write an Algorithm for Simple Union Operation?

The **Union** operation combines two disjoint sets into a single set. If i and j are the roots (representatives) of two different sets, making i the parent of j merges the two sets into one.

Algorithm UNION(i, j) { parent[j] := i }

Example: if Set 1 = {1, 2} with root 1 and Set 2 = {3, 4} with root 3, then UNION(1, 3) makes 1 the parent of 3, and the sets are merged into {1, 2, 3, 4}. The time complexity of simple union is $O(1)$

3. What is the state-space tree in backtracking?

A state-space tree is a graphical representation of all possible solutions explored during backtracking.

4. Define the Find operation in a disjoint-set data structure.

The Find operation determines the representative (root) of the set containing a given element.

5. How is a Heap Implemented?

A heap is implemented using an array representation of a complete binary tree. The parent and child nodes are accessed using index calculations, making insertion and deletion efficient.



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



6. What is the Sum of Subsets Problem?

The Sum of Subsets problem is to find all subsets of a given set whose elements add up to a specified target sum using backtracking.



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



LONG ANSWERS

1. Explain Disjoint Set Operations. Describe Union and Find algorithms with examples.

Disjoint Sets:

The disjoint sets are those that do not have any common element.

For example $S_1 = \{1, 7, 8, 9\}$ and $S_2 = \{2, 5, 10\}$, then we can say that S_1 and S_2 are two disjoint sets.

Disjoint Set Operations:

The disjoint set operations are

1. Union
2. Find

Disjoint set Union:

If S_i and S_j are two disjoint sets, then their union $S_i \cup S_j$ consists of all the elements x such that x is in S_i or S_j .

Example:

$S_1 = \{1, 7, 8, 9\}$ $S_2 = \{2, 5, 10\}$

$S_1 \cup S_2 = \{1, 2, 5, 7, 8, 9, 10\}$

FIND: The **Simple Find** operation finds the root node (representative element) of the set containing a given element.

Given the element i , find the set containing i .

Example:

$S1 = \{1,7,8,9\}$ $S2 = \{2,5,10\}$ $S3 = \{3,4,6\}$

Then,

$\text{Find}(4) = S3$ $\text{Find}(5) = S2$ $\text{Find}(7) = S1$

UNION AND FIND ALGORITHMS:

Simple Union

Definition

The Simple Union operation combines two disjoint sets into a single set by making one root the parent of another root.

Rules for Simple Union

1. Consider all elements as individual sets.
2. Initialize the parent array with -1.
3. Select two root nodes.
4. Make one root the parent of the other.
5. Update the parent array after every union.



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



Algorithm:

SimpleUnion(i, j)

```
{  
    P[j] = i;  
}
```

where

- i = root of first set
- j = root of second set

After union, i becomes the parent of j.

Example

Consider the elements

1 2 3 4 5 6

Step 1: Initial State

$P[1] = -1$

$P[2] = -1$

$P[3] = -1$

$P[4] = -1$

$P[5] = -1$

$P[6] = -1$

Tree Representation

1 2 3 4 5 6

Each element is a separate set.

Step 2: Union(1,2)

$P[2] = 1$

Tree

1

|

2

Parent Array

i	1	2	3	4	5	6
---	---	---	---	---	---	---

P[i]	-1	1	-1	-1	-1	-1
------	----	---	----	----	----	----



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



Step 3: Union(3,4)

$P[4] = 3$

Tree

3

|

4

Parent Array

i	1	2	3	4	5	6
---	---	---	---	---	---	---

P[i]	-1	1	-1	3	-1	-1
------	----	---	----	---	----	----

Step 4: Union(5,6)

$P[6] = 5$

Tree

5

|

6

Parent Array

i	1	2	3	4	5	6
---	---	---	---	---	---	---

P[i]	-1	1	-1	3	-1	5
------	----	---	----	---	----	---

Step 5: Union(1,3)

$P[3] = 1$

Tree

1

```

  /\
 2 3
  |
 4

```

Parent Array

i	1	2	3	4	5	6
P[i]	-1	1	1	3	-1	5

Step 6: Union(1,5)

P[5] = 1

Final Tree

```

  1
 /\
2 3 5
  |
 4 6

```

Final Parent Array

i	1	2	3	4	5	6
P[i]	-1	1	1	3	1	5



MALLA REDDY TECHNICAL CAMPUS
 A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
 Approved by UGC & AICTE New Delhi
 Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



Simple Find

Definition

The **Simple Find** operation finds the root node (representative element) of the set containing a given element.

Algorithm

```

Find(i)
{
  while(P[i] > 0)
  {
    i = P[i];
  }

  return i;
}

```

Rules for Simple Find

1. Start from the given node.
2. Follow parent links.
3. Continue until a node with parent -1 is reached.
4. That node is the root.
5. Return the root.

Example

Find(4)

Step 1

$i = 4$
 $P[4] = 3$
Move to parent.
 $i = 3$

Step 2

$P[3] = 1$
Move to parent.
 $i = 1$

Step 3

$P[1] = -1$
Root found.
Return 1



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



Tracing Table for Find(4)

Step	i	P[i]
1	4	3
2	3	1
3	1	-1

Therefore,
 $\text{Find}(4) = 1$

Time Complexity

Operation Complexity

Simple Union $O(1)$

Simple Find $O(n)$ (Worst Case)

2. Explain the N-Queens Problem using Backtracking. Illustrate the solution for a 4-Queens problem.

N-QUEENS PROBLEM:

The **N-Queens Problem** is a classic problem in **Backtracking** that involves placing **N queens on an $N \times N$ chessboard** such that no two queens attack each other.

Since a queen can move **horizontally, vertically, and diagonally**, the arrangement must satisfy the following conditions:

- No two queens are placed in the same row.
- No two queens are placed in the same column.
- No two queens are placed on the same diagonal.

The problem is solved using **Backtracking**, where queens are placed one row at a time. If a queen cannot be placed safely in a row, the algorithm backtracks to the previous row and tries another position. This process continues until all queens are placed successfully or all possibilities are exhausted.

Q1			



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



The second queen should not be in first row and second column. It should be placed in second row and in second, third or fourth column. If we place it in second column, both will be in same diagonal, so place it in third column.

Q1			
X	X		

Q1			
x		Q2	
	x		
Q1			
x	x	Q2	
x	x	x	x

We are unable to place queen 3 in third row, so go back to queen 2 and place it somewhere else

Q1			
----	--	--	--

			Q2
	Q3		

Q1			
			Q2
	Q3		
x	x	x	x

Now the fourth queen should be placed in 4th row and 3rd column but there will be a diagonal attack from queen 3. So go back, remove queen 3 and place it in the next column. But it is not possible, so move back to queen 2 and remove it to next column but it is not possible. So go back to queen 1 and move it to next column.



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



	Q1		
	Q1		
			Q2
		Q1	
			Q2
Q3			

	Q1		
			Q2
Q3			
		Q4	

Fig: Example of Backtrack solution to the 4-queens problem

Hence the solution of to 4-queens's problem is $x_1=2, x_2=4, x_3=1, x_4=3$, i.e first queen is placed in 2nd column, second queen is placed in 4th column and third queen is placed in first column and fourth queen is placed in third column.

Time Complexity

- Worst Case: $O(N!)$
- Space Complexity: $O(N)$



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



3. Describe the general method of Backtracking with an algorithm and write its Applications.

Definition: The backtracking algorithm is a problem-solving approach that tries out all the possible solutions and chooses the best or desired ones. Generally, it is used to solve problems that have multiple solutions.

Terminology Used in Backtracking

Root Node

Represents the initial state of the problem.

Live Node

A node that has been generated but not yet explored.

E-node (Expansion Node)

The live node currently being expanded.

Dead Node

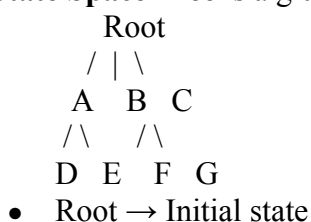
A node that cannot lead to a solution and is discarded.

Answer Node

A node representing a complete valid solution.

State Space Tree

A **State Space Tree** is a graphical representation of all possible states generated during the search.



- Internal nodes → Partial solutions
- Leaf nodes → Complete solutions
- Invalid branches are pruned

Backtracking Algorithm

Backtrack(k)

```
{
  if solution found
    print solution
  else
    {
      for each candidate x
```



MALLA REDDY TECHNICAL CAMPUS
 A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
 Approved by UGC & AICTE New Delhi
 Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



```
{
  if x is promising
    {
      include x
      Backtrack(k+1)
      remove x
    }
  }
}
```

Steps in Backtracking

1. Start from the root node.
2. Generate a candidate solution.
3. Check whether it is promising.
4. If promising, continue searching.
5. If not promising, backtrack.
6. Repeat until all solutions are explored.

Applications of Backtracking

1. N-Queens Problem
2. Graph Coloring
3. Hamiltonian Cycle
4. Sum of Subsets

4. Generate the state space tree and find the solutions for the subset sum for $N=4$, $M=50$, $W=\{10,20,30,40\}$ using backtracking approach.

Sum of Subsets Problem (Backtracking)

Definition

Given a set of positive integers $S = \{s_1, s_2, \dots, s_n\}$ and a target value d , the objective is to find all subsets of S whose sum is exactly equal to d . The problem is commonly solved using the Backtracking technique, where non-promising subsets are discarded to reduce the search space.

For the Sum of Subsets problem,

- Set: $S = \{10, 20, 30, 40\}$
- Target Sum = 50
- Let



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



- x_1 represent element 10
- x_2 represent element 20
- x_3 represent element 30
- x_4 represent element 40
- 1 = Include the element
- 0 = Exclude the element

Algorithm for Sum of Subset

1. If sum = d then
Print the subset
Return
2. If sum > d or $i = n$ then
Return
3. Include $S[i]$ in the subset
SumOfSubsets($i + 1$, sum + $S[i]$)
4. Exclude $S[i]$ from the subset
SumOfSubsets($i + 1$, sum)

Explanation

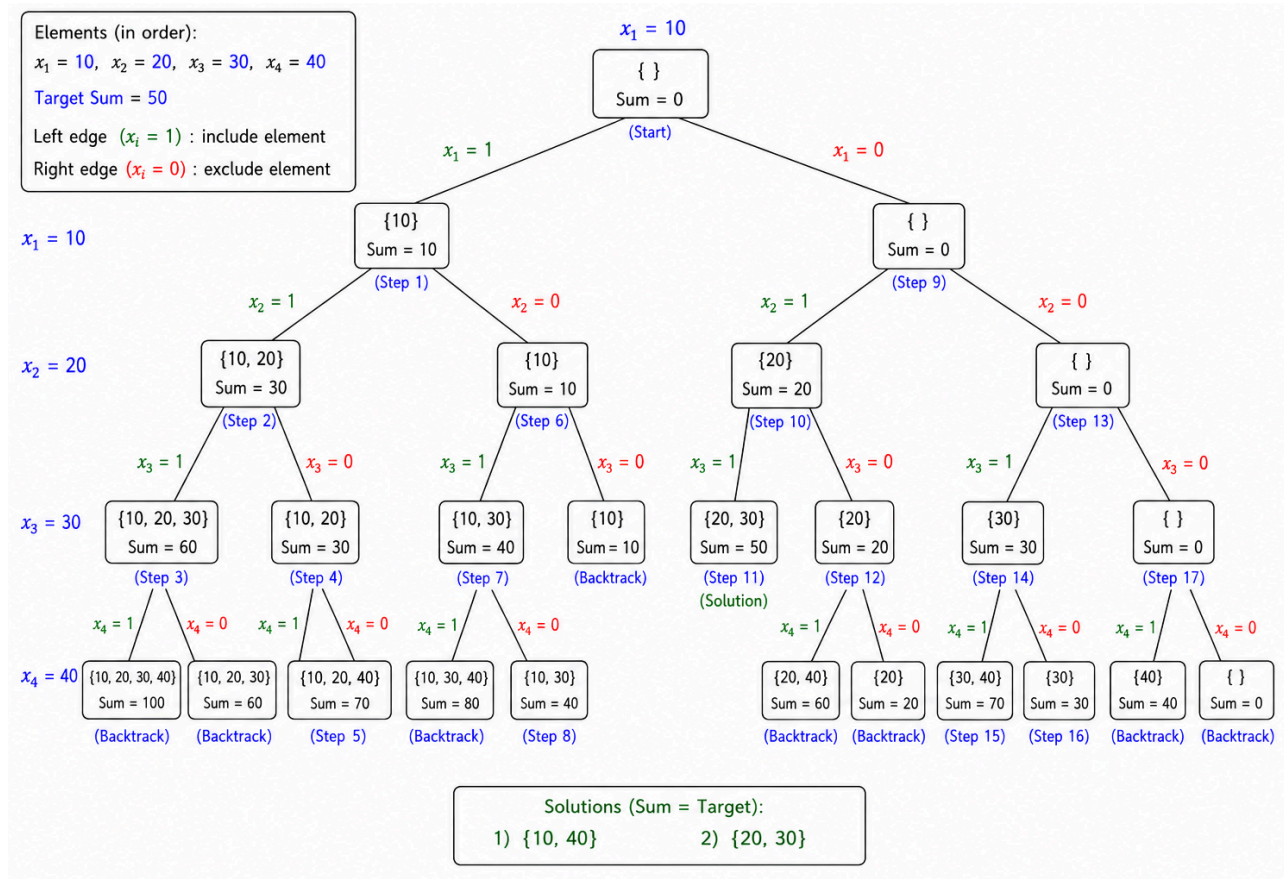
- $i \rightarrow$ Current element index.
- **sum** $\rightarrow$ Sum of elements selected so far.
- **d** $\rightarrow$ Target sum.
- **$S[i]$** $\rightarrow$ Current element being considered.

Search Table

Step	Current Subset	Sum	Action
1	{}	0	Start
2	{10}	10	Add next
3	{10,20}	30	Add next
4	{10,20,30}	60	Backtrack
5	{10,20,40}	70	Backtrack
6	{10,30}	40	Add next
7	{10,30,40}	80	Backtrack
8	{10,40}	50	Solution
9	{20}	20	Continue
10	{20,30}	50	Solution



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana





MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



5. Explain the Graph Coloring problem and solve it using the Backtracking technique.

Definition:

Let G be a graph and m be a given positive integer. We want to discover whether the nodes of G can be colored so that no two adjacent nodes have the same color, using only m colors. This is called the m -colorability decision problem.

Notes:

- If d is the degree of the graph, it can be colored with $d+1$ colors.
- The smallest value of m required is called the chromatic number.

General Backtracking Algorithm

$mColoring(k)$

while (true)

 NextValue(k)

 if $x[k] == 0$
 return

 if $k == n$
 print($x[1...n]$)

 else
 $mColoring(k+1)$

Algorithm NextValue(k)

1. Repeat

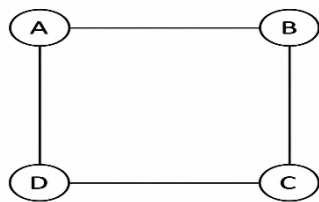
2. Assign next color to vertex k.
3. If no color is available, return.
4. Compare the color of vertex k with all adjacent vertices.
5. If any adjacent vertex has the same color, try the next color.
6. Otherwise, accept the color and return.
7. Repeat until a valid color is obtained.



MALLA REDDY TECHNICAL CAMPUS
 A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
 Approved by UGC & AICTE New Delhi
 Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



Example Graph



Vertices: A, B, C, D

Edges:

- A — B
- B — C
- C — D
- D — A

Adjacency of Vertices

- A is adjacent to B and D
- B is adjacent to A and C
- C is adjacent to B and D
- D is adjacent to A and C

Adjacency Matrix

	V1	V2	V3	V4
V1	0	1	0	1
V2	1	0	1	0
V3	0	1	0	1
V4	1	0	1	0

Coloring Process

Initially

$x = [0, 0, 0, 0]$

where

- $0 \rightarrow$ No color assigned
- $1 \rightarrow$ Red
- $2 \rightarrow$ Green
- $3 \rightarrow$ Blue

Step 1

Assign **V1 (A) = Color 1 (Red)**

$x = [1, 0, 0, 0]$



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



Step 2

Assign V2 (B)

V2 is adjacent to V1.

- Color 1 (Red) ✗ (Same as V1)
- Color 2 (Green) ✓
- Color 3 (Blue) ✓

Choose Color 2 (Green).

$x = [1, 2, 0, 0]$

Step 3

Assign V3 (C)

V3 is adjacent only to V2.

- Color 1 (Red) ✓
- Color 2 (Green) ✗ (Same as V2)
- Color 3 (Blue) ✓

Choose Color 1 (Red) (first valid color).

$x = [1, 2, 1, 0]$

Step 4

Assign V4 (D)

V4 is adjacent to V1 and V3.

- Color 1 (Red) ✗ (Same as V1 and V3)
- Color 2 (Green) ✓
- Color 3 (Blue) ✓

Choose Color 2 (Green).

$x = [1, 2, 1, 2]$

Final Coloring

Vertex	Color Number	Color Name
V1 (A)	1	Red
V2 (B)	2	Green
V3 (C)	1	Red
V4 (D)	2	Green

Final Color Assignment

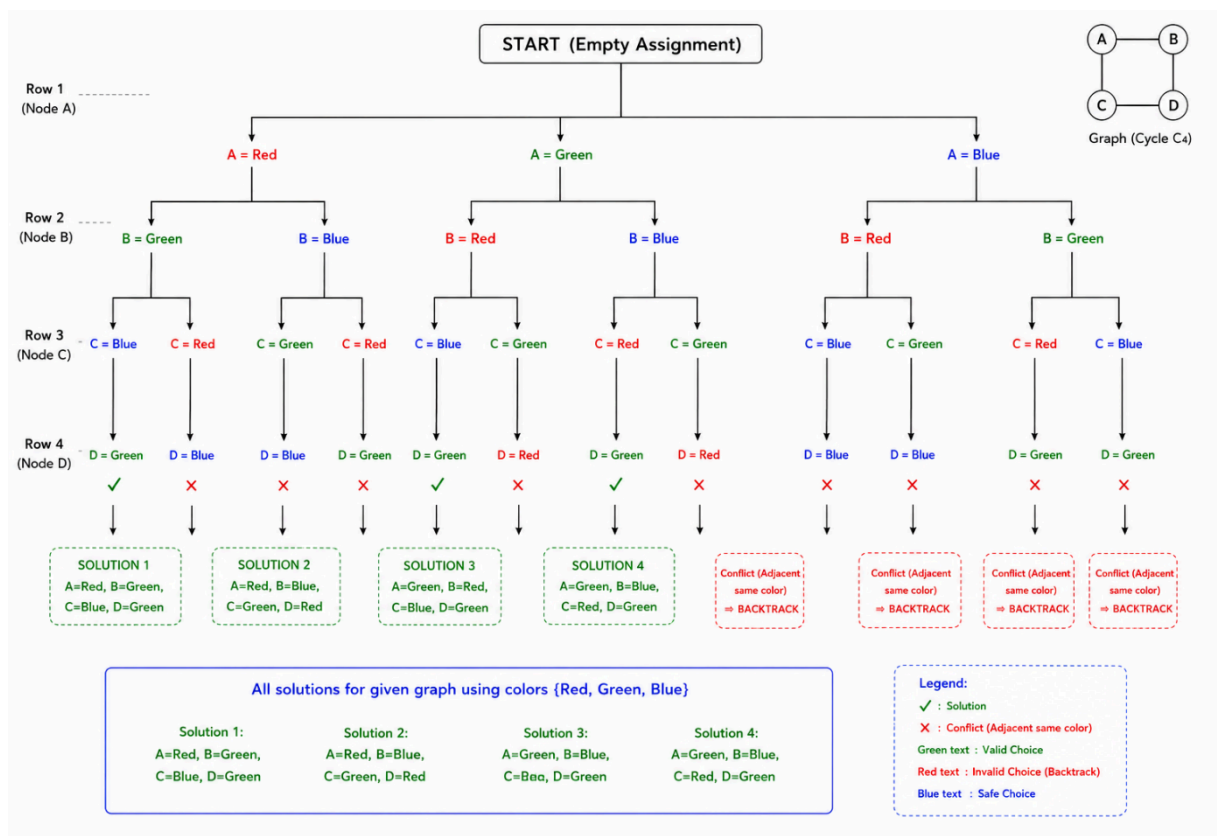
V1 (A) → Color 1 (Red)
 V2 (B) → Color 2 (Green)
 V3 (C) → Color 1 (Red)
 V4 (D) → Color 2 (Green)



MALLA REDDY TECHNICAL CAMPUS
 A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
 Deemed to be University
 Approved by UGC & AICTE New Delhi
 Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



State Space Tree



Time Complexity

- **Worst Case:** $O(m^n)O(mn)$
 - nn = number of vertices
 - mm = number of colors

Space Complexity

- $O(n)$ for the recursion stack and color array.



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



6. What is a Hamiltonian Cycle? Explain the Backtracking solution for finding Hamiltonian Cycles in a graph.

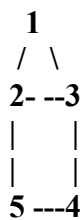
A **Hamiltonian Cycle** is a cycle in a graph that **visits every vertex exactly once and returns to the starting vertex**.

- Each vertex is visited only once.
- The first and last vertices are the same.
- Not every graph contains a Hamiltonian cycle.

Problem Statement

Given an undirected graph $G(V, E)$, determine whether there exists a Hamiltonian cycle.

Example Graph



One Hamiltonian Cycle is:

$1 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 1$

Idea of Backtracking

- Start from the first vertex.
- Add one vertex at a time to the path.
- Before adding a vertex, check whether it is valid.
- If a wrong choice is made, **go back (backtrack)** and try another vertex.
- Continue until all vertices are included in the cycle.

Conditions to Add a Vertex

A vertex can be added only if:

1. It is connected to the previous vertex.
2. It has not been used before.
3. If it is the last vertex, it must connect back to the starting vertex.



MALLA REDDY TECHNICAL CAMPUS
 A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
 Approved by UGC & AICTE New Delhi
 Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



ALGORITHM:

Hamiltonian(k)

If all vertices are included

Print the cycle

Else

Try every vertex

If the vertex is valid

Add it to the path

Call Hamiltonian(k+1)

Remove it (Backtrack)

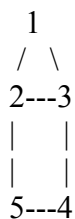
Validity Check (Easy)

IsValid(vertex)

1. Check whether it is connected to the previous vertex.
2. Check whether it is already used.
3. If it is the last vertex,
check whether it connects to the first vertex.

Step-by-Step Example

Graph:



Step 1

Start from vertex 1

Path = [1]

Step 2

Choose 2

Path = [1,2]

Step 3

Choose 5

Path = [1,2,5]

Step 4

Choose 4

Path = [1,2,5,4]



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



Step 5

Choose 3

Path = [1,2,5,4,3]

Step 6

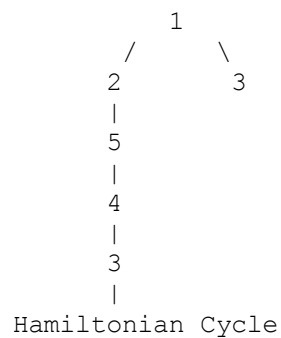
Check whether 3 is connected to 1.

Yes ✓

So the Hamiltonian Cycle is:

$1 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 1$

State Space Tree



Time Complexity

- Worst Case: $O(n!)$
- Space Complexity: $O(n)$



MALLA REDDY TECHNICAL CAMPUS

A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH

Deemed to be University

Approved by UGC & AICTE New Delhi

Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana

